Ca' Foscari University of Venice

Development Methodology

Matteo Zanioli

matteo.zanioli@unive.it

Data Models in System Integration

- A data model is a conceptual framework that organizes and defines how data is represented, structured, stored, and manipulated within a system. It provides a formal way to describe data objects, their relationships, attributes, and rules for interaction, ensuring consistency and accuracy in how data is managed.
- Data models play a crucial role in System Integration by defining the structure, relationships, and rules governing data within and across systems.
- In integration scenarios, they serve as a common framework that enables disparate systems to share, transform, and understand data effectively.

Data Model in System Integration

- *Standardization*: Create a shared vocabulary for data representation across different systems.
- *Data Consistency*: Ensure that data exchanged between systems remains accurate and uniform.
- *Interoperability*: Allow systems with different architectures or formats to work together.
- *Optimization*: Enhance performance by tailoring data structures to the needs of integrated applications.

- Example: In an e-commerce ecosystem, a customer's profile might exist in multiple systems (CRM, billing, shipping). A unified data model ensures all systems interpret and update the profile consistently.

- Understanding the three levels of data modeling helps manage data effectively in integration projects.

Conceptual Data Models

- Purpose: Defines what data is represented and how different entities relate to each other, independent of any technical implementation.

- Key Features:
  - High-level representation (e.g., customers, products, orders).
  - Focuses on business requirements rather than technical details.

- Example:
  - Entities: `Customer`, `Order`, `Product`.
  - Relationships: `Customer` places multiple `Orders`, each `Order` contains one or more `Products`.

Logic Data Models

- Purpose: Specifies how the data will be structured and organized, translating business concepts into detailed data definitions.

- Key Features:
  - Includes attributes (e.g., `CustomerName`, `OrderDate`).
  - Defines primary keys, foreign keys, and relationships.
  - Technology-agnostic but more detailed than the conceptual model.

- Example:
  - Table for `Customer` with attributes: `CustomerID`, `Name`, `Email`.
  - Relationship: `Order.CustomerID` → `Customer.CustomerID`.

Physical Data Models

- Purpose: Focuses on the actual implementation of the data model in a specific database system.

- Key Features:
  - Includes database-specific constraints and structures (indexes, storage types).
  - Optimized for performance and integration needs.

- Example:SQL schema with data types:

```
CREATE TABLE Customer (
        CustomerID INT PRIMARY KEY,
        Name VARCHAR(255),
        Email VARCHAR(255)
);
```

Data Models Summary

Aspect	Conteptual	Logical	Physical
Focus	High-level entities	Data structure and attributes	Database-specific implementation
Audience	Business stakeholders	Analysts, architects	DBAs, developers
Technology	Independent	Independent	Dependent

Normalization and Denormalization

- In system integration, data often comes from multiple sources. *Normalization* and *denormalization* are techniques used to structure this data effectively.

Normalization

- Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. This is achieved by dividing data into related tables and defining relationships between them, following a set of rules known as normal forms. The primary goals of normalization are:
  - To eliminate duplicate data.
  - To ensure that data dependencies are logical and minimal.
  - To make the database easier to maintain and update.
- Normalization typically involves structuring data into smaller, well-organized tables and linking them using relationships, such as primary and foreign keys.

Denormalization

- Denormalization is the process of intentionally introducing redundancy into a database by combining tables or duplicating data.
- This is done to optimize query performance, reduce join operations, and speed up data retrieval in scenarios where performance is critical, such as in data warehousing or high-traffic applications.
- While denormalization can improve read performance, it often comes at the cost of increased storage requirements and more complex updates, as changes to redundant data must be synchronized across the system.
- It is generally used when the benefits of faster reads outweigh the drawbacks of potential inconsistencies and additional maintenance.

Normalization vs Denormalization

Normalization

- Goals:
  - Reduce redundancy.
  - Improve data consistency.
  - Ensure efficient updates and deletions.
- Process:
  - 1NF: Eliminate duplicate rows, ensure atomicity of data
  - 2NF: Remove partial dependencies on a composite key.
  - 3NF: Eliminate transitive dependencies.
- Example:
  - A `Customer` table should not contain order details; instead, these details should reside in an `Order` table linked by `CustomerID`.

Denormalization

- Goals:
  - Optimize read performance.
  - Simplify data access in integration scenarios.
- Example:
  - Combining `Customer` and `Order` tables into a single table in a read-heavy reporting system to avoid costly joins.

Centralized vs. Distributed

- Integrating systems involves coordinating data flow and processes. There are two main approaches: **orchestration** and **choreography**.

Centralized Orchestation

- Definition: A central system (or orchestrator) manages interactions between systems.
- How It Works: The orchestrator sends requests, collects responses, and ensures workflows are executed in order.
- Advantages:
  - Easier to monitor and debug workflows.
  - Centralized control simplifies implementation.
- Disadvantages:
  - Single point of failure.
  - Scalability challenges as the number of integrated systems grows.
- Example: A centralized ERP system initiates and monitors a product shipping workflow across inventory, billing, and logistics systems.

Distributed Choreography

- Definition: Each system involved in the process is responsible for deciding when and how to interact with others.
- How It Works: Systems communicate using events rather than relying on a central controller.
- Advantages:
  - High scalability.
  - Decoupled architecture increases resilience.
- Disadvantages:
  - Harder to monitor and debug due to lack of centralized oversight.
  - Requires well-defined event standards.
- Example:
  - In an e-commerce platform, the `Order Service` emits an event when an order is placed, triggering independent actions in the `Payment`, `Inventory`, and `Shipping` services.

Centralized vs. Distributed

Aspect	Orchestration	Choreography
Control	Centralized	Distributed
Complexity	Lower initially, grows with scale	Higher but scalable
Resilience	Single point of failure	Fault-tolerant
Use Case	Linear workflows	Event-driven, dynamic processes

Ca' Foscari University of Venice

Data Governance

- In system integration, managing and securing data is critical. With systems exchanging vast amounts of sensitive information, data governance ensures that data is accurate, consistent, and secure, while complying with regulations.
- Data governance refers to the processes, policies, and standards established to manage the availability, integrity, and security of data across an organization.
- In system integration, it becomes essential due to:
  - Data Silos: Integration often involves disparate systems with varying data standards.
  - Compliance Requirements: Laws such as GDPR mandate strict data handling protocols.
  - Data Quality: Ensuring data consistency and accuracy across systems.
  - Security Risks: Protecting sensitive data from breaches or unauthorized access.

Ca' Foscari University of Venice

Data Governance and GDPR

- What is GDPR?
  - The General Data Protection Regulation (GDPR) is a European Union law that governs the collection, processing, and storage of personal data, with a focus on protecting individual privacy.
- Key GDPR Principles
  - Data Minimization: Only collect and process data that is necessary.
  - Accountability: Organizations must demonstrate compliance.
  - Right to Access and Erasure: Individuals can request access to or deletion of their personal data.
  - Data Protection by Design: Systems must be designed with security and privacy as core principles.
- Implications for System Integration
  - Consent Management: Integrated systems must track and respect user consent for data use.
  - Data Transfers: Cross-border integrations need safeguards for international data transfers.
  - Data Retention Policies: Integrated systems must enforce retention and deletion schedules consistently.
  - Audit Trails: Integrated systems must record all data-related activities to ensure compliance.
- Example: In an integrated CRM and marketing system, GDPR compliance requires tracking when and how user consent was obtained, ensuring that data shared with third-party systems adheres to user permissions.

Other Regulations

- CCPA (California Consumer Privacy Act): Similar to GDPR but focuses on the rights of California residents.
- HIPAA (Health Insurance Portability and Accountability Act): Governs health-related data in the US.
- ISO 27001: An international standard for information security management.

Data Lineage

- Definition: Data lineage refers to tracking the origin, transformation, and movement of data across systems.
- Why is Data Lineage Important?
  - Transparency: Provides visibility into how data flows between systems.
  - Compliance: Demonstrates how data has been processed and transferred to meet regulatory requirements.
  - Troubleshooting: Identifies errors or inconsistencies in data pipelines.
- Example: In a supply chain system, data lineage could trace the journey of a product ID from the manufacturer's ERP system to the retailer's sales platform.

Audit Trail

- Definition: An audit trail is a record of all activities and changes performed on data within a system.
- Key Features of an Audit Trail:
  - Event Tracking: Logs who accessed or modified data, when, and what changes were made.
  - Immutable Records: Ensures logs cannot be altered or deleted.
  - Scalability: Handles logs across multiple systems in real-time.
- Benefits of Audit Trails:
  - Security: Detects unauthorized access or data breaches.
  - Compliance: Provides proof of regulatory adherence.
  - Operational Insights: Highlights inefficiencies or issues in processes.
- Example: In a banking system, an audit trail logs when customer information is accessed or updated, which is essential for both security and compliance.

Challenges in Data Governance

- Data Standardization: Harmonizing data formats and structures across systems.
- Cross-System Security: Ensuring consistent security policies in heterogeneous environments.
- Scalability: Maintaining governance as the number of integrated systems grows.
- Real-Time Tracking: Managing lineage and audit trails for real-time data exchanges.

Strategies for Effective Data Governance

- Define Clear Policies: Establish guidelines for data handling, retention, and sharing.
- Implement Role-Based Access Control (RBAC): Restrict data access to authorized users only.
- Use Data Encryption: Protect data during transmission and storage.
- Automate Monitoring: Leverage tools to monitor data lineage and generate audit logs automatically.
- Invest in Data Integration Tools: Use platforms that support governance features (e.g., MuleSoft, Talend, Apache NiFi).

Data Inconsistencies

- System integration often involves connecting disparate systems with differing data formats, standards, and interpretations.
- Inconsistencies between these systems are one of the primary challenges, particularly when integrating legacy systems with modern platforms.
- Addressing these inconsistencies requires identifying potential issues and employing strategies to ensure coherence and accuracy in data exchange.

Ca' Foscari University of Venice

Common Issues in Data Mapping

- Different Data Formats:
  - Systems may represent the same data differently.
  - Example: Date formats (DD/MM/YYYY vs. MM-DD-YYYY), numerical separators (1,000 vs. 1.000).
- Unit of Measure Discrepancies:
  - Inconsistent use of units leads to incorrect calculations or interpretations.
  - Example: Weight (kilograms vs. pounds), distance (meters vs. feet).
- Field Name and Structure Mismatches:
  - Fields in one system may not exist in another or may have different names and structures.
  - Example: CustomerID in one system vs. ClientID in another.
- Data Type Conflicts:
  - Data types might differ, leading to errors during transformation.
  - Example: Storing currency as float in one system and decimal in another.
- Semantic Differences:
  - The same field may have different meanings across systems.
  - Example: Status in one system may indicate "active/inactive," while in another, it could represent a shipping state like "shipped/delivered."

Strategies to Ensure Consistency - ETL

- ETL processes are widely used to standardize data across systems.
- Extract: Retrieve data from source systems.
- Transform: Apply transformations to align formats, units, and semantics.
  - Example Transformations:
    - Convert date formats.
    - Map fields using a lookup table (ClientID → CustomerID).
    - Standardize units (e.g., convert all weights to kilograms).
- Load: Load the cleaned and transformed data into the target system.

- Advantages:
  - Centralized control over data transformation.
  - Supports batch processing for large data volumes.
- Challenges:
  - Limited for real-time integration.

Strategies to Ensure Consistency – Data Federation

- Definition: Allows querying and retrieving data from multiple systems in real time without physically consolidating it.
- How It Works:
  - A middleware layer interprets and standardizes data at query time.
  - Virtual views present unified data to users or systems.
- Example:A federated query fetches customer data from both a CRM and an e-commerce platform, presenting a combined view.
- Advantages:
  - No need to replicate or store data centrally.
  - Enables real-time data access.
- Challenges:
  - Performance may degrade with high query volumes.
  - Relies on consistent network connectivity.

Strategies to Ensure Consistency – MDM

- Definition: Centralized management of critical business data (master data) to ensure consistency and accuracy across systems.
- Key Features:
  - Creates a "golden record" as the single source of truth for each entity (e.g., customers, products).
  - Synchronizes updates across integrated systems.
- Example:
  - A global retailer ensures all systems reference the same standardized product catalog via MDM.
- Advantages:
  - Maintains consistency even in complex, multi-system integrations.
  - Reduces redundancy and errors.
- Challenges:
  - High implementation cost and complexity.
  - Requires strong organizational commitment to governance.

Ca' Foscari University of Venice

Best practice for Data Mapping Issues

- Data Profiling:
  - Analyze source data to identify discrepancies and inconsistencies before integration.
- Field Mapping Tools:
  - Use automated tools to map fields between systems (e.g., Talend, Informatica).
- Schema Validation:
  - Validate schema compatibility between source and target systems.
- Data Transformation Rules:
  - Define rules for handling discrepancies, such as default values for missing fields or conversion logic for mismatched units.
- Testing and Validation:
  - Test data exchanges rigorously to identify edge cases or anomalies.
- Error Handling Mechanisms:
  - Implement fallback strategies (e.g., retry logic, logging errors for manual review).

Schema Evolution in System Integration

- Schema Evolution refers to the process of managing changes in the structure or design of a data schema—such as a database schema, API schema, or message schema—over time without disrupting the functionality of the systems that rely on it.
- This concept is essential in dynamic environments where systems grow, requirements change, and new features are introduced, necessitating updates to how data is structured, stored, or exchanged. Schema evolution ensures compatibility, data integrity, and seamless integration between systems by adopting strategies like versioning, incremental updates, and backward/forward compatibility.

Schema Evolution in System Integration

- In dynamic environments, schema evolution is critical as systems change over time. Whether it's introducing new fields, modifying data types, or deprecating unused elements, evolving schemas must preserve the integrity of integrations and avoid disruptions to dependent systems.

- Challenges
  - Backward Compatibility:
    - Ensuring changes do not break existing integrations.
    - Example: Adding a required field may cause failures in systems unaware of it.
  - Forward Compatibility:
    - Preparing current systems to work with future schema changes.
    - Example: Ignoring unknown fields added by future systems.
  - Dependency Management:
    - Systems may rely on outdated schemas or transformations.
  - Performance Overhead:
    - Schema changes can introduce latency in data processing due to added transformation layers.
  - Data Migration:
    - Aligning historical data with new schema definitions.

Best practice - Versioning

- Versioned APIs:
  - Maintain different versions of an API to support old and new clients.
  - Example: /v1/customers vs. /v2/customers.
- Versioned Schemas:
  - Use schema registries (e.g., Apache Avro, JSON Schema) to track and validate schema versions.
  - Example: customer_schema_v1.json, customer_schema_v2.json.
- Advantages:
  - Ensures backward compatibility.
  - Allows incremental updates.

Best practice – Extensible Schema Designs

- Design schemas to accommodate future changes without breaking existing systems.
- Optional Fields: Use nullable fields or optional attributes for new data.
- Open Schema Design: Allow unknown fields to be ignored (e.g., JSON or XML extensibility).

```
{
        "customerID": "12345",
        "name": "John Doe",
        "email": "john.doe@example.com",
        "extraInfo": {}
}
```

Best practice – Embrace Schema Registries

- Use tools like Apache Avro, Apache Kafka Schema Registry, or Protocol Buffers.
- Automate schema validation.
- Manage schema compatibility policies (backward, forward, or full compatibility).
- Schema Compatibility Levels:
  - Backward Compatible: New schema works with data written using the old schema.
  - Forward Compatible: Old schema works with data written using the new schema.
  - Full Compatible: Ensures both backward and forward compatibility.

Best practice – Incremental Data Migration

- Gradually migrate data to align with the new schema:
  - Dual Writes: Write to both old and new schemas during the transition.
  - Data Replication Pipelines: Use tools like Apache NiFi or AWS Glue to transform and migrate data.

- Example:
  - A product catalog system introduces a new `discount` field. Historical data is updated in batches, while new records include the field immediately.

Best practice – Implement Transformation Layers

- Use middleware or ETL processes to translate between old and new schemas dynamically.
- Example: Map `fullName` in the old schema to `firstName` and `lastName` in the new schema.

Techniques for Managing Schema Evolution

- Schema Evolution in Databases
  - Adding Columns:
    - Safest change as existing data remains valid.
  - Renaming Columns:
    - Introduce aliases in views or transformation layers to bridge the old and new schema.
  - Dropping Columns:
    - Mark columns as deprecated first and phase them out gradually.
- Schema Evolution in APIs
  - Deprecate Features Gradually:
    - Notify users of upcoming changes with clear timelines.
  - Feature Toggles:
    - Allow clients to opt into new features or schema changes.
- Event-Driven Architectures
  - Evolving Event Schemas:
    - Use versioned event types or include metadata to describe schema versions.
  - Handling Unknown Fields:
    - Consumers should ignore unexpected fields to avoid breakage.

Schema Evolution

- Managing schema evolution is essential to maintain seamless system integrations over time.
- By adopting best practices like versioning, extensible designs, and incremental migrations, organizations can minimize disruptions and ensure robust, future-proof integrations.

Interoperability and Standardization

- Interoperability and standardization are essential for enabling diverse systems to communicate effectively. They ensure that data can be exchanged, understood, and processed consistently, regardless of platform or technology differences.
- Key concepts:
  - Interoperability: The ability of different systems or components to exchange information and use it meaningfully.
  - Standardization: The use of agreed-upon formats, protocols, and practices to facilitate interoperability.

Common Data Exchange Formats - JSON

- JSON – JavaScript Object Notation
- Lightweight, human-readable, and widely used for web APIs.
- Strengths:
  - Simplicity and readability.
  - Native support in modern programming languages.

```
{
        "customerID": "12345",
        "name": "John Doe",
        "email": "john.doe@example.com",
}
```

Common Data Exchange Formats - XML

- XML - eXtensible Markup Language
- Highly structured and extensible, often used in enterprise applications.
- Strengths:
  - Supports complex hierarchies and metadata.
  - Schema validation via DTDs or XSDs.

```
<customer>
        <id>123</id>
        <name>John Black</name>
        <email>john.black@example.com</email>
</customer>
```

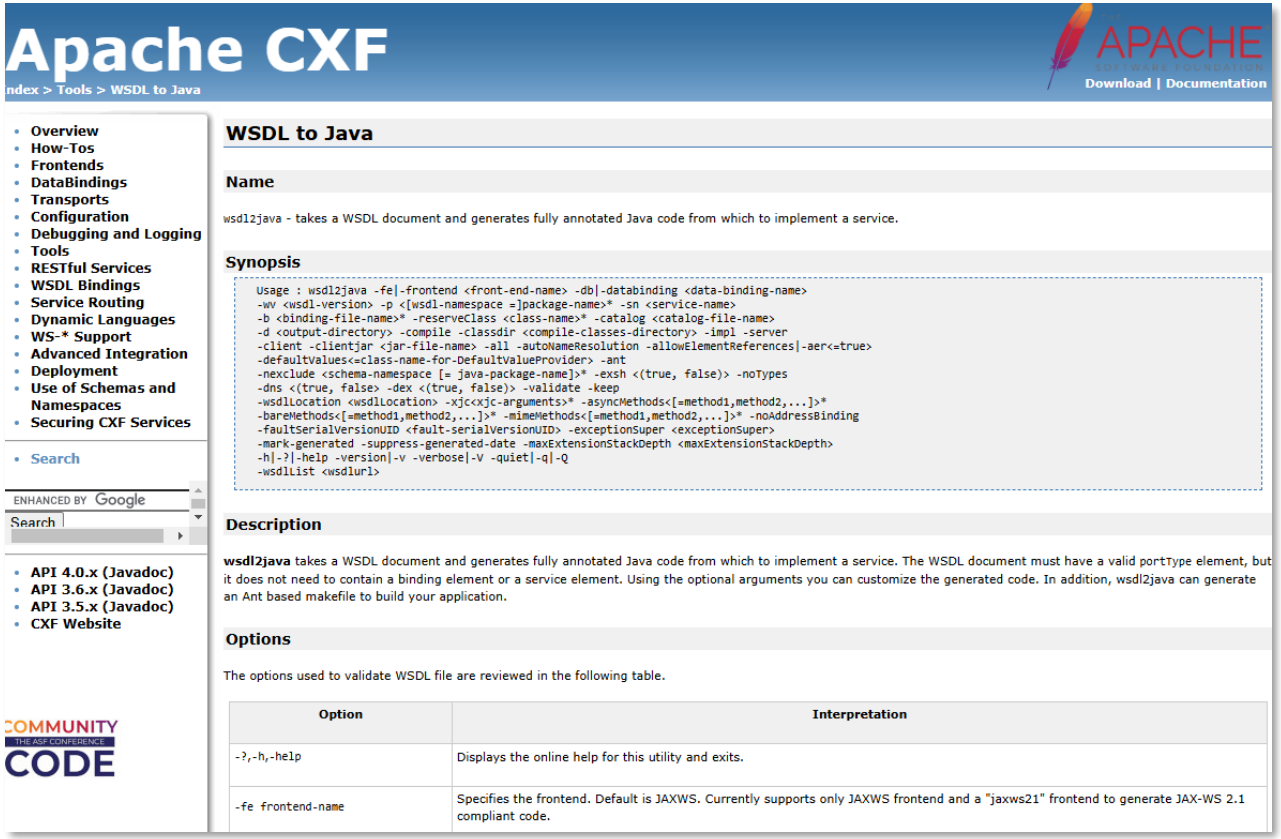
Common Data Exchange Formats - Toon

- Toon – Typed Object-Oriented Notation
- It combines the simplicity of JSON with the extensibility and type safety of object-oriented paradigms.
- Strenght:
  - Supports complex data structures and strong typing, reducing errors during integration.
  - Readable syntax that facilitates both manual editing and automated processing.
  - Extensible, allowing for custom data types and schemas to suit diverse integration needs.

```
hikes[3]{id,name,distanceKm,elevationGain,companion,was
Sunny}:
  1,Blue Lake Trail,7.5,320,ana,true
  2,Ridge Overlook,9.2,540,luis,false
  3,Wildflower Loop,5.1,180,sam,true
```

Common Data Exchange Formats - SOAP

- SOAP – Simple Object Access Protocol
- A protocol that uses XML for exchanging structured information in web services.
- Strengths:
  - Robust security and error-handling capabilities.
  - Standardized approach for enterprise-level integrations.



Apache CXF

ndex > Tools > WSDL to Java

APACHE SOFTWARE FOUNDATION

Download | Documentation

- Overview
- How-Tos
- Frontends
- DataBindings
- Transports
- Configuration
- Debugging and Logging
- Tools
- RESTful Services
- WSDL Bindings
- Service Routing
- Dynamic Languages
- WS-* Support
- Advanced Integration
- Deployment
- Use of Schemas and Namespaces
- Securing CXF Services

- Search

ENHANCED BY Google

Search

- API 4.0.x (Javadoc)
- API 3.6.x (Javadoc)
- API 3.5.x (Javadoc)
- CXF Website

COMMUNITY THE ASF CONFERENCE CODE

WSDL to Java

Name

wsdl2java - takes a WSDL document and generates fully annotated Java code from which to implement a service.

Synopsis

```
Usage : wsdl2java -fe|-frontend <front-end-name> -db|-databinding <data-binding-name>
-wv <wsdl-version> -p <[wsdl-namespace =]package-name>* -sn <service-name>
-b <binding-file-name>* -reserveClass <class-name>* -catalog <catalog-file-name>
-d <output-directory> -compile -classdir <compile-classes-directory> -impl -server
-client -clientjar <jar-file-name> -all -autoNameResolution -allowElementReferences|-aer<=true>
-defaultValues<=class-name-for-DefaultValueProvider> -ant
-nexclude <schema-namespace [= java-package-name]>* -exsh <(true, false)> -noTypes
-dns <(true, false> -dex <(true, false)> -validate -keep
-wsdlLocation <wsdlLocation> -xjc<xjc-arguments>* -asyncMethods<[=method1,method2,...]>*
-bareMethods<[=method1,method2,...]>* -mimeMethods<[=method1,method2,...]>* -noAddressBinding
-faultSerialVersionUID <fault-serialVersionUID> -exceptionSuper <exceptionSuper>
-mark-generated -suppress-generated-date -maxExtensionStackDepth <maxExtensionStackDepth>
-h|-?|-help -version|-v -verbose|-V -quiet|-q|-Q
-wsdlList <wsdlurl>
```

Description

wsdl2java takes a WSDL document and generates fully annotated Java code from which to implement a service. The WSDL document must have a valid portType element, but it does not need to contain a binding element or a service element. Using the optional arguments you can customize the generated code. In addition, wsdl2java can generate an Ant based makefile to build your application.

Options

The options used to validate WSDL file are reviewed in the following table.

Option	Interpretation
-?,-h,-help	Displays the online help for this utility and exits.
-fe frontend-name	Specifies the frontend. Default is JAXWS. Currently supports only JAXWS frontend and a "jaxws21" frontend to generate JAX-WS 2.1 compliant code.

Common Data Exchange Formats - REST

- REST – Representational State Transfer
- Architectural style for designing networked applications, often using JSON or XML.
- Strengths:
  - Simplicity and scalability.
  - Stateless communication and widespread adoption.

Industrial Integration Standards

- EDI (Electronic Data Interchange):
  - Standard for exchanging business documents like purchase orders and invoices.
  - Features:
    - Uses defined formats (e.g., ANSI X12, EDIFACT).
    - Reduces manual data entry and errors.
  - Use Case: Retail and supply chain integrations.

- OPC-UA (Open Platform Communications - Unified Architecture):
  - Standard for interoperability in industrial automation.
  - Features:
    - Platform-independent and secure communication.
    - Supports real-time data exchange.

- HL7 (Health Level Seven):
  - Standard for exchanging clinical and administrative data in healthcare.
  - Use Case: Integration between hospital management systems and diagnostic tools.

Ca' Foscari University of Venice

Industrial Integration Standards

- Benefits of Standardization
  - Reduced Complexity: Standard formats and protocols simplify integrations and reduce custom development.
  - Enhanced Compatibility: Standardization ensures systems can communicate seamlessly, even if built by different vendors.
  - Improved Efficiency: Streamlined data exchanges reduce errors and speed up processes.
  - Scalability: Standardized integrations are easier to scale and extend as business needs evolve.

- Challenges in Interoperability and Standardization
  - Legacy Systems :Older systems may lack support for modern standards, requiring additional middleware.
  - Adherence to Standards: Variations in how standards are implemented can lead to compatibility issues.
  - Security Risks: Standard protocols need robust security measures to prevent data breaches.
  - Industry-Specific Standards: Vertical standards may not align perfectly with broader integration efforts.

Ca' Foscari University of Venice

Industrial Integration Standards

- Case Study: Supply Chain Integration
- Scenario: A retailer integrates its inventory management system with supplier systems using EDI.
- Challenges:
  - Suppliers use different file formats and protocols.
  - High volume of transactions requires reliable messaging.
- Solution:
  - Use EDI for standard document exchange (e.g., purchase orders in ANSI X12 format).
  - Deploy a middleware layer for protocol conversion and error handling.
- Outcome:
  - Reduced manual data entry.
  - Faster order processing and fewer errors.
- Conclusions
  - Interoperability and standardization are the cornerstones of efficient system integration. By adopting universal data formats, messaging protocols, and industry-specific standards, organizations can create seamless, scalable, and secure integrations that adapt to evolving business needs.